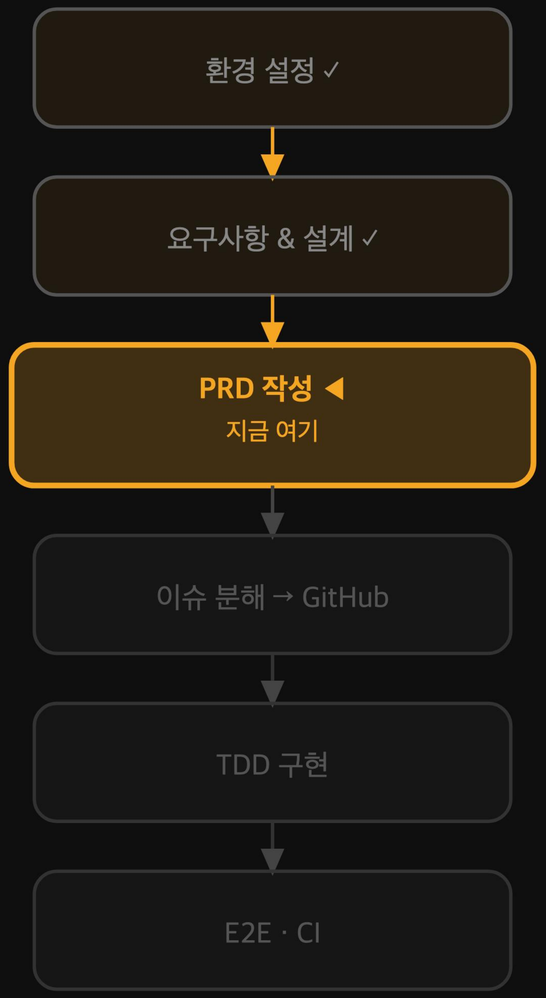
6강. PRD 작성 & 기술 결정(ADR)

단일 기준 문서 · ADR 4요소 · Out of Scope

지금 여기 있습니다

PRD 작성 단계



요구사항은 완성됐다. 이제 무엇을?

spec-fixed.md가 있다.

그런데 이걸 어떻게 구현의 단일 기준 문서로 만들 것인가?

이번 강의 목표

PRD 작성

spec과 기술 결정을 하나의 기준 문서로 통합해 이후 모든 이슈 분해의 출발점으로 쓴다

기술 결정(ADR)

PRD의 "기술 결정" 섹션을 ADR 4요소로 작성해 "왜 이렇게 했는가"를 보존한다

Out of Scope

AI의 범위 초과 구현을 막는 울타리를 세운다

PRD — 모든 이슈의 단일 기준 문서

PRD(Product Requirements Document)

spec-fixed.md(무엇) + 기술 결정(어떤 구조 위에) → **하나의 문서**

"이 기능에 대해 궁금하면 여기만 보면 된다"

PRD도 진화해왔다

- **1980~90년대 — 워터폴 시대**
  수백 페이지짜리 "계약 문서". Microsoft·IBM 등이 코딩 전 모든 디테일을 못 박으려 함
- **2000년대 — 애자일의 등장**
  "PRD 무용론"과 "가벼운 PRD"로 갈림. 긴 계약서는 변화에 못 따라간다는 비판
- **현재 — 살아있는 단일 기준 문서**
  Notion·Confluence·리포지토리 안에서 짧고 맥락 있는 형태로 진화

이 강의는 "가벼운 PRD" 계열 — 짧지만 결정의 "왜"까지 담는다

PRD의 5가지 구성 요소

개요

기능의 목적과 범위

사용자 스토리

사용자 관점의 흐름

기술 결정

어떤 구조 위에 올릴 것인가

Out of Scope

이번에 하지 않을 것

용어 정의

spec-fixed.md 용어를 이어받아 통일

가장 어려운 건 "기술 결정" 섹션

개요·사용자 스토리는 spec-fixed.md에서 가져온다.

Out of Scope은 "하지 않을 것"을 적으면 된다.

기술 결정은? 이제 정해야 한다.

기술 결정은 복리로 작용한다

요구사항이 바뀌면 코드 일부만 수정하면 된다.

기술 결정이 바뀌면 코드 전체를 재구성해야 한다.

앞 이슈의 기술 결정 하나가 뒤 이슈들의 난이도를 전부 바꾼다

AI에게 기술 결정을 맡기면 안 되는 이유

팀의 기술 수준

과도한 설계는 오히려 부담이 된다

향후 로드맵

6개월 뒤 추가될 기능을 AI는 모른다

비즈니스 제약

배포 일정, 유지보수 인력을 AI는 모른다

ADR(Architecture Decision Record)

- **2010년 전후 — 문제 상황**
  과도한 문서화 속에서도 "왜 이 기술을 썼지?"를 물으면 아무도 답할 수 없었다
- **Michael Nygard의 제안 (2011)**
  "Documenting Architecture Decisions"에서 가볍고 실용적인 형식 — 원형은 Status · Context · Decision · Consequences
- **MADR의 확장 (2010년대 후반)**
  Nygard 원형에 "대안 비교(Considered Options)"를 더해 지금 널리 쓰는 형태로 진화
- **핵심 철학**
  "결정 자체"에 집중 — 설계 결과물이 아니라 결정을 내린 맥락과 trade-off를 남기는 것

코드만 보면 "어떻게"는 알 수 있다. "왜"는 알 수 없다.

ADR의 4요소

Context
이 결정이 필요한 맥락 (당시의 제약·요구)

Decision
무엇을 선택했는가

Alternatives
함께 검토한 대안과 거부 이유

Consequences
결정 이후의 결과 (장점·단점·달라지는 것 모두)

PRD 안에 ADR이 산다

별도 ADR 파일을 만들지 않는다.

PRD의 "기술 결정" 섹션을 ADR 4요소로 쓴다.

파일 하나로 단일 진실 원천. 결정이 많아지면 그때 분리 (YAGNI).

3가지 안을 같은 기준으로 비교

비교 기준	안 A	안 B	안 C
데이터 구조	Note 안 배열	별도 테이블	배열 + 캐시
API 변경지점	Note CRUD 확장	Tag API 신설	Note CRUD 확장
상태관리 변경지점	Note store 내부	Tag store 신설	Note store + 캐시
즉시 저장 동작	단순	복잡	중간
컴포넌트 구조	기존 재사용	신규 다수	기존 재사용
기존 패턴 일관성	★★★★★	★★★	★★★★
테스트 용이성	높음	중간	높음

비교해야 좋은 Alternatives 섹션이 나온다

Out of Scope이 가장 중요하다

"태그 색상 커스터마이징은 이번 범위 밖"이라고 명시하지 않으면

AI가 알아서 구현해버린다.

하지 않을 것의 명시가 할 것의 명시만큼 중요하다

실습

PRD 뼈대

spec-fixed 기반 생성 (기술 결정은 TODO)

3안 비교 & 선택

AI에게 7가지 기준으로 비교 요청

ADR 4요소 작성

Context · Decision · Alternatives · Consequences

Out of Scope & 커밋

범위 검증 후 커밋